

02 - Analisi del Dominio e Proposte architettoniche

Alessandro Gardini

`alessandro.gardini7@studio.unibo.it`

Lorenzo Rossi

`lorenzo.rossi50@studio.unibo.it`

Tirocizio - Università di Bologna - Campus di Cesena

10 giugno 2026

1 Analisi Dominio

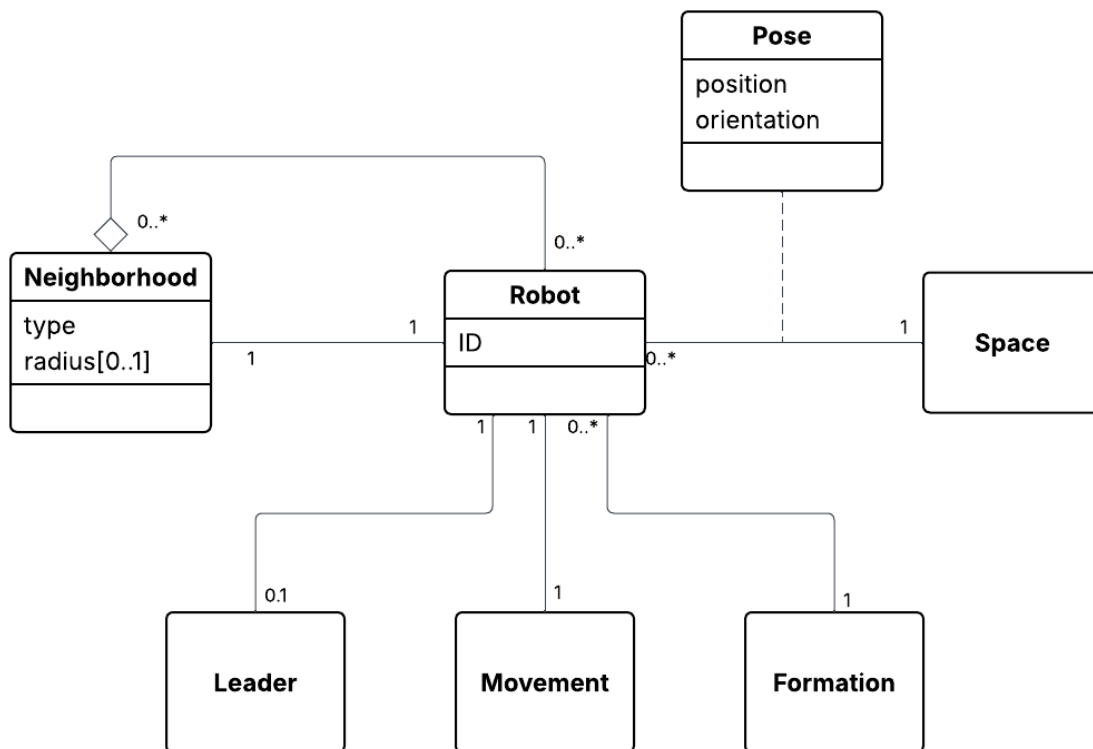


Figura 1: Schema entità

Questo schema modella le entità del dominio, in un determinato snapshot temporale, rimanendo volutamente agnostico rispetto all'implementazione, senza legarsi a tecnologie o scelte architetturali specifiche. Segue la spiegazione di ogni entità e della sua rappresentazione specifica nel progetto.

1.1 Space

Lo *Space* nel nostro dominio rappresenta lo spazio bidimensionale dove i robot possono muoversi.

1.2 Robot

Il *Robot* è l'entità principale del dominio. È univocamente rappresentato da un identificatore, un numero intero.

1.3 Pose

Pose rappresenta la coppia *Position* e *Orientation*. *Position* rappresenta una coppia di coordinate x e y reali. *Orientation* rappresenta invece la direzione in cui è rivolto il *robot*, espressa in radianti.

1.4 Neighborhood

La *Neighborhood* rappresenta tutti i *Robot* raggiungibili da un determinato *Robot*. Può essere calcolata in due modi: **FULL** e **RADIUS**. **FULL** calcola tutti i *Robot* come nella *Neighborhood* di tutti. **RADIUS** calcola la *Neighborhood* come i *Robot* presenti ad una distanza massima definita da **radius**. Dato un *Robot*, cioè il suo identificatore è quindi dato un insieme di *Robot* definiti dai propri identificatori.

1.5 Leader

Il *Leader* è il ruolo assunto dal capo del gruppo di *Robot*. Definito dall'identificatore del *Robot* che assume quel ruolo.

1.6 Formation

La *Formation* rappresenta come si devono disporre i *Robot*. È definita da un **valore**, una **lable** e da **parametri** specifici per quel tipo di formazione.

1.7 Movement

Il *Movement* è lo spostamento che il *Robot* deve compiere per raggiungere la posizione a lui assegnata data la *Formation*. Implementato come due valori, uno per la ruota destra e una per la sinistra.

2 Proposte architetturali

2.1 Proposte per Off-loading

2.1.1 Sistema Dinamico

La realizzazione di un sistema dinamico, in grado di prendere decisioni in autonomia, si basa sull'introduzione di un modulo di controllo apposito. Il modulo si occupa di monitorare continuamente lo stato del sistema e, qualora rilevi la necessità di redistribuire parte del carico computazionale, di inviare apposite richieste ai robot disponibili. Ciascun robot può accettare o rifiutare la richiesta in base alle proprie condizioni operative. In caso di accettazione, il modulo provvede ad aggiornare lo stato complessivo del sistema e ad informare tutte le parti coinvolte della variazione avvenuta.

2.1.2 Decisioni decentralizzate

In questa soluzione, le decisioni riguardanti le tattiche di off-loading vengono effettuate in modo distribuito. Un modulo pensato per le decisioni o entità esterne al sistema, come una dashboard operata dall'utente o il drone stesso, invieranno le proprie decisioni direttamente ai singoli moduli interessati e si dovranno occupare di leggere e interpretare i rispettivi messaggi e coordinare le conseguenti azioni.

Nell'esempio 2, le decisioni vengono prese da una dashboard controllata dall'operatore: questa invia una richiesta al drone, il quale, dopo aver verificato la propria disponibilità e le condizioni operative, risponde in modo coerente accettando o rifiutando il cambio di configurazione. Il Robot deve contattare tutti i moduli interessati di questo cambiamento.

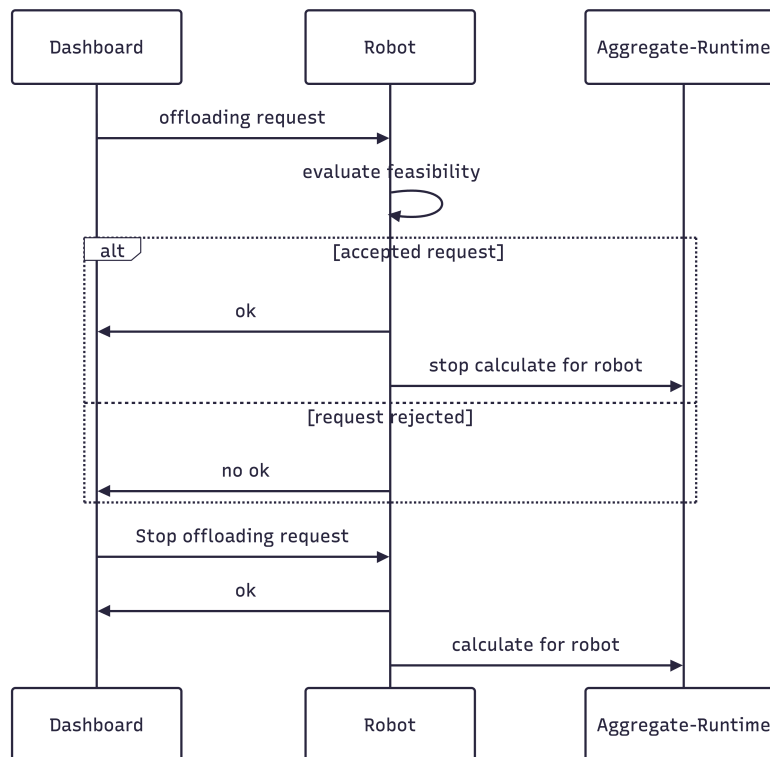


Figura 2: Sistema di decisioni decentralizzato

2.1.3 Modulo di decisioni centralizzato

Questa soluzione si basa sull'introduzione di un modulo aggiuntivo dedicato, denominato *offloading-manager*, al quale viene delegata la gestione delle decisioni e l'attuazione delle strategie di off-loading e la gestione delle richieste da parte di altri moduli.

Questo modulo permette così un punto di accesso centralizzato per le decisioni di off-loading: attori esterni possono interagire direttamente per indicare quali robot debbano farsi carico di determinate funzionalità, evitando di esporre tutti i moduli relativi.

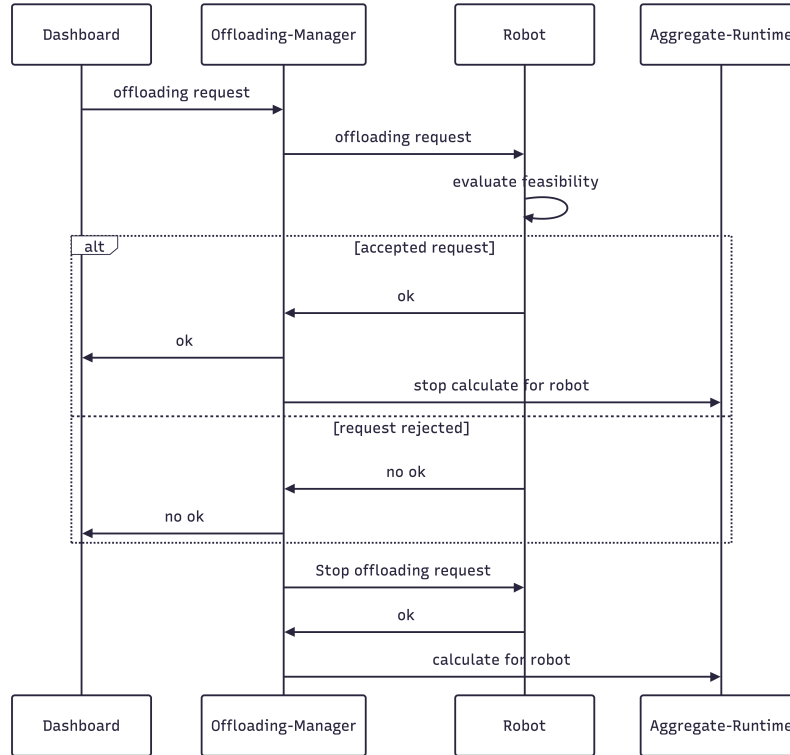


Figura 3: Sistema centralizzato per l'offloading

2.1.4 Comparazione delle due soluzioni

La soluzione **decentralizzata**, realizzando collegamenti diretti tra i moduli interessati risulta più semplice da implementare, realizzando il modulo per l'offloading solamente come agente di controllo.

La soluzione con **modulo centralizzato** (*offloading-manager*) che si occupa sia del controllo del sistema che da interfaccia per altri agenti per le decisioni, rende la complessità implementativa iniziale maggiore, permettendo però valutazioni sulle decisioni esterne e una più facile manutenibilità dovuta alla presenza di un unico accesso per il sistema.

2.2 Proposte di estensione oltre a MQTT

In questo paragrafo vengono presentate tre metodologie per gestire l'integrazione di *robot* che comunicano con il sistema tramite protocolli diversi da MQTT.

2.2.1 Server di traduzione diretta in MQTT

Per garantire la copertura verso gli altri *robot*, viene istanziato un server che espone una serie di protocolli all'esterno. Tramite appositi endpoint, ciascun *robot* può iscriversi a un determinato *topic*: al momento della sottoscrizione, il server memorizza le informazioni necessarie per contattare il *robot* e registra l'interesse dichiarato, iscrivendosi contestualmente al *topic* corrispondente presso il broker. Alla ricezione di un messaggio da parte del broker, il server provvede a re-instradarlo verso il *robot* sottoscritto. Nel caso in cui un *robot* debba invece pubblicare un messaggio su un *topic*, questo viene inoltrato direttamente al broker interno. Ogni altro *robot* che intende comunicare con MQTT invece continuerebbe ad utilizzare direttamente il broker interno.

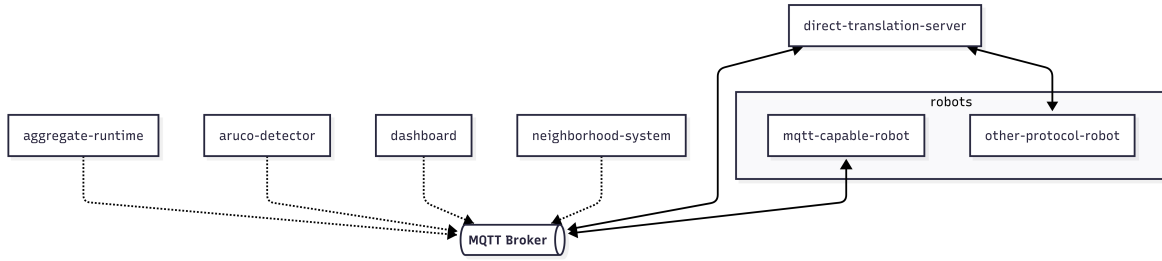


Figura 4: Server di traduzione diretta in MQTT - Grafico

2.2.2 Server cache che espone uno snapshot del sistema

In questa seconda soluzione, il server dedicato alla comunicazione esterna si sottoscrive a tutti i *topic* del broker, costruendo e mantenendo aggiornato uno snapshot dell'intero stato del sistema. Tuttavia, questo approccio implica un cambiamento nel paradigma di comunicazione: i *robot* connessi a tale servizio non potrebbero più sfruttare il modello ad eventi offerto da MQTT, ma sarebbero costretti ad adottare una strategia di *polling* per interrogare periodicamente il server. La comunicazione nel verso opposto, ovvero dall'esterno verso l'interno, avviene invece in modo diretto: il server si occupa semplicemente di inoltrare i messaggi ricevuti al broker interno, senza alcuna elaborazione aggiuntiva. I *robot* interni che comunicano tramite MQTT continuerebbero ad interagire direttamente con il broker senza alcuna modifica.

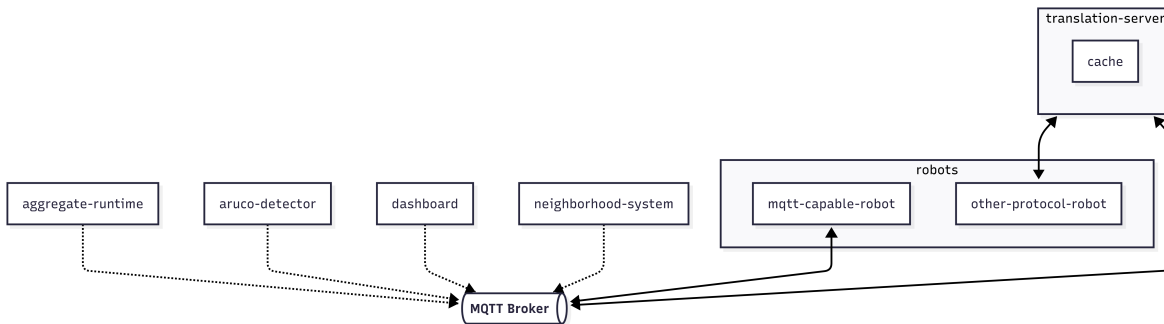


Figura 5: Server cache che espone uno snapshot del sistema - Grafico

2.2.3 Completo redesign architetturale verso un modello REST

Una terza soluzione prevede un ripensamento più radicale dell'intera architettura. Il broker centrale verrebbe sostituito da un server REST, il quale assumerebbe la responsabilità di mantenere uno snapshot aggiornato dello stato del sistema: qualsiasi componente che necessiti di leggere o modificare lo stato

dovrebbe interfacciarsi esclusivamente con questo servizio. La gestione delle connessioni esterne, indipendentemente dal protocollo utilizzato — MQTT, HTTP o CoAP — sarebbe invece delegata a un servizio secondario dedicato. Questo approccio risulta essere il più oneroso in termini di migrazione, poiché ogni componente interno dovrebbe essere riscritto nella parte dedicata alla gestione delle connessioni, e comporta la perdita della natura reattiva della comunicazione, con possibili ripercussioni sulla latenza — sebbene quest’ultima possa essere mitigata tramite **WebSocket** laddove necessario. Tuttavia, sul lungo periodo questa soluzione offre i maggiori vantaggi in termini di manutenibilità: sostituendo il contratto implicito dei *topic* MQTT con *API* esplicite e documentabili, si semplifica la comprensione del sistema.

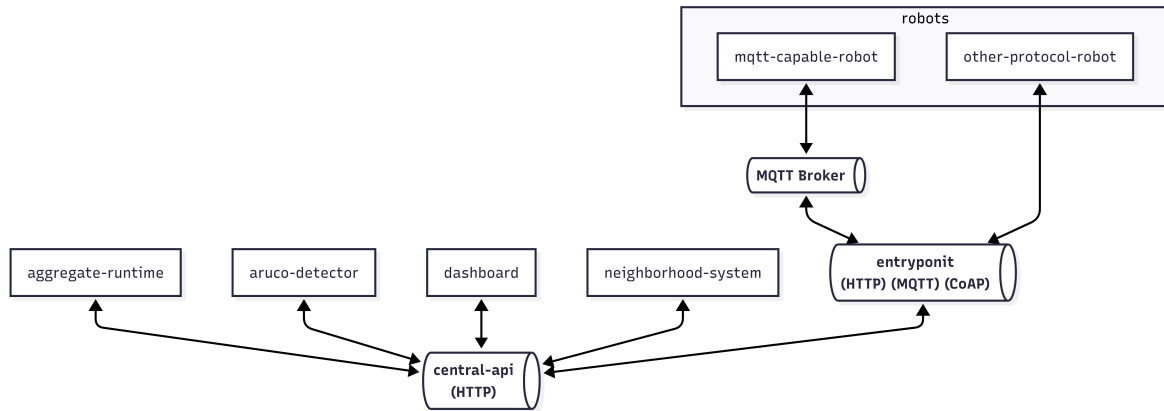


Figura 6: Completo redesign architettuale verso un modello REST - Grafico

2.2.4 Confronto delle proposte

La prima soluzione è quella che meglio si integra con l’architettura esistente, richiedendo modifiche minime e preservando il modello ad eventi. La seconda risulta la meno appetibile dato che comporterebbe comunque un cambio di paradigma dalla soluzione attuale senza però migliorare la leggibilità del sistema e la manutenibilità. La terza è la più onerosa in termini di tempo, ma è anche quella che garantisce la maggiore manutenibilità a lungo termine. API esplicite e documentate potrebbero ridurre significativamente la barriera d’ingresso per ulteriori aggiunte.